Costruttori e modificatori

Introduzione

Nel corso delle lezioni precedenti sono stati introdotti i **concetti fondamentali della programmazione orientata agli oggetti in Java**, analizzando il ruolo delle classi, degli oggetti, dei costruttori e dei metodi.
Abbiamo approfondito l'uso della keyword `this`, il meccanismo di **overloading** e **overriding**, e studiato in dettaglio la classe `String`, mettendone in evidenza le caratteristiche principali come l'immutabilità, i metodi di lettura, ricerca, confronto e parsing, oltre alle differenze con `StringBuffer` e `StringBuilder`.
Successivamente sono stati introdotti i **modificatori di accesso**, che regolano la visibilità di classi e membri, evidenziando le differenze tra `public`, `private`, `protected` e visibilità di default.
A partire da queste basi, la lezione attuale introduce nuovi concetti chiave per la progettazione delle classi:
i **modificatori** `static` **e** `final`, il loro significato e il loro impatto sul comportamento di attributi, metodi e costruttori.
Questi elementi permettono un controllo più preciso sul ciclo di vita degli oggetti e sulla struttura delle classi, risultando fondamentali per scrivere codice robusto, chiaro e manutenibile.

Esempio: classe Impiegato

Per spiegare in modo concreto l'uso dei **modificatori** in Java, partiamo dalla modellizzazione di una semplice classe `Impiegato`.

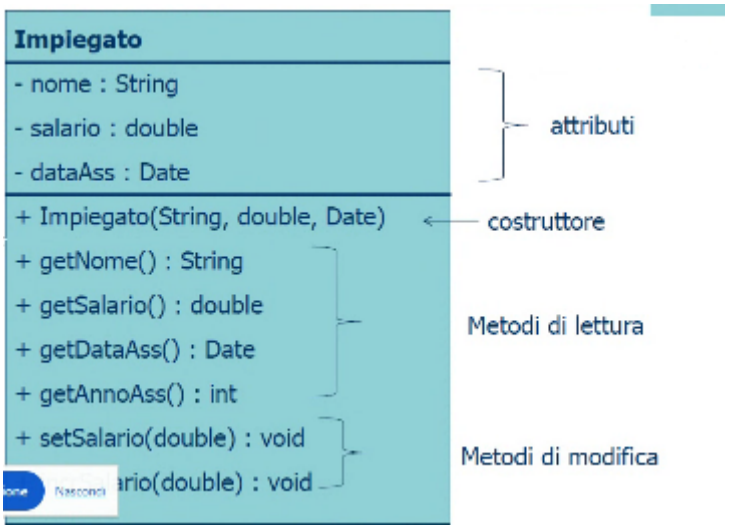
Vogliamo rappresentare un **impiegato** che possiede i seguenti attributi fondamentali:

- `nome : String`
- `salario : double`
- `dataAssunzione : Date`

Nel diagramma UML la classe viene rappresentata secondo le seguenti scelte progettuali:

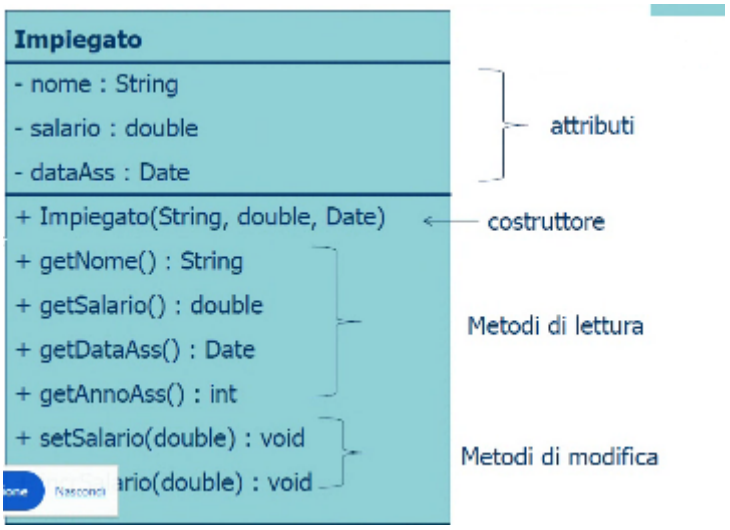
- **attributi privati**, per rispettare il principio di incapsulamento
- **metodi pubblici**, che costituiscono l'interfaccia della classe

- un **costruttore**, responsabile dell'inizializzazione dell'oggetto
- un metodo per **modificare il salario** dell'impiegato



ImpiegatoJava.png

Questa classe verrà utilizzata come riferimento per introdurre e comprendere il ruolo dei principali **modificatori Java** (`private` , `public` , `final` , `static`) e il loro impatto sulla progettazione delle classi.



ImpiegatoJava.png

Costruttori

Come in Python e in JavaScript, il **costruttore** è:

- **un metodo speciale che ha il compito di inizializzare un oggetto nel momento in cui viene creato.**
  In Java, ogni classe possiede **almeno un costruttore**, esplicito o implicito.

Proprietà dei costruttori

Un costruttore in Java ha alcune caratteristiche fondamentali:

- **Il nome coincide sempre con il nome della classe**
  - **Questo permette al compilatore di riconoscerlo come costruttore.**
- **Non si specifica alcun tipo di ritorno**
  - Il costruttore non restituisce valori: **il suo scopo è creare e inizializzare l'oggetto.**
- **Ha il compito di valorizzare tutti gli attributi della classe**
  - **Un oggetto, dopo l'esecuzione del costruttore, deve trovarsi in uno stato valido.**

Esempio costruttore in Java:

Java

```java
public Impiegato(String n, double s, Date d) {
    nome = n;
    salario = s;
    dataAss = d;
}
```

In questo esempio:

- il costruttore si chiama `Impiegato`, come la classe
- riceve come parametri i valori iniziali degli attributi
- assegna tali valori agli attributi dell'oggetto appena creato

Quando viene eseguita un'istruzione come:

Java

```java
Impiegato i = new Impiegato("Mario Rossi", 1500.0, new Date());
```

avviene quanto segue:

1. **viene allocata la memoria per l'oggetto** `Impiegato`
2. **viene invocato il costruttore**
3. **gli attributi** `nome`**,** `salario` **e** `dataAss` **vengono inizializzati**

In questo modo l'oggetto nasce **già pronto all'uso**, senza stati intermedi incoerenti.

La keyword `this`

La keyword `this` viene utilizzata all'interno dei metodi (e in particolare nei costruttori) **per riferirsi all'oggetto chiamante, cioè all'istanza corrente della classe.**

In altre parole:

- `this` **rappresenta l'oggetto su cui il metodo è stato invocato**
- **permette di accedere esplicitamente agli attributi e ai metodi dell'oggetto**

Problema dello *shadowing*

In Java è possibile dichiarare i **parametri di un metodo con lo stesso nome degli attributi della classe**.
Questa situazione prende il nome di **shadowing** (ombreggiamento):

- **il parametro locale "nasconde" l'attributo della classe**
- **senza** `this`**, il compilatore fa riferimento alla variabile locale, non all'attributo**

Esempio tipico nel costruttore:

Java

```java
public Impiegato(String nome, double salario, Date dataAss) {
    this.nome = nome;
    this.salario = salario;
    this.dataAss = dataAss;
}
```

Qui accade quanto segue:

- `nome` **(a destra) è il parametro del costruttore**
- ==== `this.nome` (a sinistra) è l'**attributo dell'oggetto**
- **l'assegnazione serve a copiare il valore del parametro nell'attributo**

Senza l'uso di `this`, l'istruzione:

Java

```java
nome = nome;
```

assegnerebbe il parametro a sé stesso, **senza modificare l'attributo**, rendendo il costruttore inefficace.

Quando `this` è necessario

- `this` **è obbligatorio quando esiste shadowing tra attributi e parametri**
- **è facoltativo quando i nomi sono diversi, ma spesso viene usato comunque per chiarezza**

Esempio senza shadowing:

Java

```java
public Impiegato(String n, double s, Date d) {
    nome = n;
    salario = s;
    dataAss = d;
}
```

Qui `this` non è necessario perché non c'è ambiguità, ma molti programmatori preferiscono comunque scrivere:

Java

```java
this.nome = n;
```

per rendere esplicito che si sta lavorando sugli attributi dell'oggetto.

Significato concettuale di `this`

Dal punto di vista concettuale, `this` rafforza l'idea che:

- ogni oggetto possiede **il proprio stato**
- il metodo sta operando **su quell'istanza specifica**
- l'oggetto "sa chi è" mentre esegue il codice

Questo concetto diventerà ancora più importante quando si parlerà di:

- **overloading dei costruttori** (`this(...)`)
- **ereditarietà** (`super`)
- **polimorfismo**

Valori di default degli attributi

In Java, **ogni attributo di una classe viene sempre inizializzato, anche quando il programmatore non assegna esplicitamente un valore nel costruttore.**

Se nel costruttore **non si valorizza un attributo**, il compilatore (più precisamente la JVM) assegna automaticamente un **valore di default**, che dipende dal tipo dell'attributo:

- `null` **per tutti i tipi oggetto**
  - (incluse le `String`)
- `false` **per il tipo** `boolean`
- `'\u0000'` **(carattere nullo) per il tipo** `char`
- `0` **per tutti i tipi numerici (** `int`**,** `double`**,** `long`**, ecc.)**

Esempio:

Java

```java
public class Impiegato {
    private String nome;
    private double salario;
    private boolean assunto;
}
```

Se istanzio un oggetto senza inizializzare esplicitamente gli attributi:

Java

```java
Impiegato i = new Impiegato();
```

gli attributi assumeranno automaticamente questi valori:

- `nome → null`
- `salario → 0.0`
- `assunto → false`

Questo comportamento è **diverso da molte altre variabili locali**, che invece **devono essere inizializzate manualmente** prima dell'uso.

Il costruttore di default

Come già anticipato nella lezione sulle classi in Java,
se non viene scritto alcun costruttore all'interno della classe, il compilatore aggiunge automaticamente un costruttore di default.

Il costruttore di default ha le seguenti caratteristiche:

- **non ha parametri**
- **inizializza tutti gli attributi con i valori di default**
- esiste solo nel **bytecode (** `.class` **)**, non è visibile nel file `.java`

Esempio:

Java

```java
public class Impiegato {
    private String nome;
    private double salario;
}
```

Anche se non è scritto, il compilatore genera implicitamente qualcosa di equivalente a:

Java

```java
public Impiegato() {
    nome = null;
    salario = 0.0;
}
```

Overloading dei costruttori

Riprendo il concetto di overloading definito nella lezione sulle classi, in Java è possibile definire **più costruttori all'interno della stessa classe**, purché abbiano **una lista di parametri diversa**.
Questa caratteristica prende il nome di **overloading dei costruttori**.

Le regole sono le stesse viste per l'overloading dei metodi:

- **il nome del costruttore è sempre uguale al nome della classe;**
- **ciò che distingue un costruttore da un altro è la lista dei parametri (numero, tipo o ordine);**
- **non esiste un tipo di ritorno, quindi questo aspetto non è rilevante.**

Esempio di overloading del costruttore

Java

```java
public class Impiegato {

    private String nome;
    private double salario;
    private Date dataAss;

    // Costruttore completo
    public Impiegato(String nome, double salario, Date dataAss) {
        this.nome = nome;
        this.salario = salario;
        this.dataAss = dataAss;
    }

    // Costruttore con nome e salario
    public Impiegato(String nome, double salario) {
        this.nome = nome;
        this.salario = salario;
        this.dataAss = null;
    }

    // Costruttore con solo il nome
    public Impiegato(String nome) {
        this.nome = nome;
        this.salario = 0;
        this.dataAss = null;
    }
}
```

In questo esempio:

- tutti i costruttori hanno **lo stesso nome** (`Impiegato`);
- ogni costruttore ha una **firma diversa**;
- l'oggetto può essere creato in modi differenti, a seconda delle informazioni disponibili.

🔗 Collegamento con i valori di default

L'overloading dei costruttori è spesso utilizzato per:

- consentire diverse modalità di creazione dell'oggetto;
- inizializzare solo una parte degli attributi;
- lasciare alcuni attributi ai **valori di default** quando non sono disponibili informazioni complete.

Perché mettere più costruttori nella stessa classe

L'idea chiave è che **non tutti gli oggetti devono essere creati nello stesso modo**, perché spesso le informazioni disponibili al momento della creazione sono **diverse**.
In altre parole, un oggetto della classe che vogliamo rappresentare nel dominio può nascere in modi differenti, e sta a noi decidere **come modellarlo e inizializzarlo**.
Prendiamo l'esempio della classe `Impiegato`:

Java

```java
public class Impiegato {

 private String nome;
private double salario;

 public Impiegato() {
     nome = "sconosciuto";
     salario = 0;
 }

 public Impiegato(String nome) {
      this.nome = nome;
     salario = 0;
 }

  public Impiegato(String nome, double salario) {
     this.nome = nome;
     this.salario = salario;
  }
}

```

Qui abbiamo tre modi diversi di creare un impiegato:

1. **Senza alcuna informazione** (`Impiegato()`):
   Utile se non conosci ancora nulla dell'impiegato. Viene creato un oggetto "di default" con valori iniziali sicuri.
2. **Con solo il nome** (`Impiegato(String nome)`):
   - **Utile se hai almeno il nome, ma non il salario**.
   - **L'oggetto viene inizializzato in uno stato valido senza dover inserire dati inesistenti**.
3. **Con nome e salario** (`Impiegato(String nome, double salario)`):
   - **Utile quando hai tutte le informazioni fin dall'inizio.**

✓ **Vantaggi Principali**

- **Oggetti sempre in uno stato valido:**
  - **il costruttore obbliga a inizializzare gli attributi secondo le regole della classe.**
- **Flessibilità:**
  - **puoi creare un oggetto con informazioni parziali o complete senza scrivere codice duplicato**.
- **Overloading:**
  - **tutti i costruttori hanno lo stesso nome della classe, ma parametri diversi → il compilatore sa quale chiamare**.
- **Leggibilità:**
  - **chi legge il codice capisce subito quali dati servono per creare un certo oggetto.**

"Teorema" delle classi (costruttori)

Alla luce di quanto visto sui **costruttori**, sui **valori di default** e sull'**overloading**, è possibile enunciare alcune proprietà generali che valgono per **tutte le classi Java**.

- **Ogni classe possiede almeno un costruttore.**
  - Se il programmatore **non definisce esplicitamente alcun costruttore**, il compilatore ne inserisce automaticamente uno **di default**:
    - **senza parametri;**

    - **che inizializza tutti gli attributi ai rispettivi valori di default (null, 0, false, ecc.), come visto in precedenza.**
- **Una classe può definire più costruttori**.
    - **Questo permette di creare oggetti della stessa classe in modi diversi, a seconda delle informazioni disponibili al momento dell'istanziazione.**
- **I costruttori di una classe sono sempre in overloading**.
    - Come per i metodi, anche i costruttori:
        - **hanno lo stesso nome (quello della classe);**
        - **si distinguono esclusivamente per la lista dei parametri.**
            - **Il tipo di ritorno non è rilevante, poiché i costruttori non ne dichiarano uno esplicito.**

Esempio: overloading dei costruttori nella classe `Date`

Un esempio concreto di quanto descritto è fornito dalla classe `java.util.Date`, che mette a disposizione più costruttori:

Java

```java
Date()
Date(int year, int month, int day)
Date(int year, int month, int day, int hours, int minutes)
```

In questo caso:

- **tutti i costruttori hanno lo stesso nome (`Date`);**
- **ciascun costruttore accetta una diversa combinazione di parametri;**
- **il programmatore può scegliere il costruttore più adatto in base al livello di dettaglio richiesto.**

Note sui costruttori

Dopo aver visto che una classe può definire **più costruttori in overloading**, è importante chiarire alcune regole fondamentali sul loro utilizzo in Java.

- **Un oggetto può essere costruito una sola volta**.
    - Il costruttore viene invocato esclusivamente al momento della **creazione dell'oggetto** tramite l'operatore `new`.

    - Una volta che l'oggetto è stato istanziato, **non è possibile richiamare nuovamente un costruttore** sullo stesso oggetto: l'inizializzazione avviene una sola volta.
- **Un costruttore può chiamare un altro costruttore della stessa classe**.
    - Java mette a disposizione questo meccanismo, detto **cross calling dei costruttori**, tramite la keyword `this`.
    - In questo caso `this(...)` non fa riferimento all'oggetto, ma viene utilizzato per:
    - riutilizzare codice di inizializzazione già scritto;
    - evitare duplicazioni;
    - centralizzare la logica di costruzione dell'oggetto.

> **La chiamata a `this(...)` deve essere sempre la prima istruzione del costruttore.**

Cross calling dei costruttori: esempio con `Impiegato`

Riprendendo la classe `Impiegato`, consideriamo un costruttore "completo" che inizializza tutti gli attributi:

Java

```java
public Impiegato(String nome, double salario, Date dataAss){
    this.nome = nome;
    this.salario = salario;
    this.dataAss = dataAss;
}
```

Supponiamo ora di voler permettere la creazione di un `Impiegato` senza specificare la data di assunzione, assumendo implicitamente la **data odierna**. Invece di riscrivere la logica di inizializzazione, possiamo delegarla al costruttore precedente:

Java

```java
// Costruttore di Impiegato con data odierna
public Impiegato(String nome, double salario){
    this(nome, salario, new Date());
}
```

In questo caso:

- **il secondo costruttore non inizializza direttamente gli attributi;**
- **richiama il primo costruttore passando come terzo parametro** `new Date()`**;**
- **la logica di inizializzazione rimane concentrata in un unico punto.**

> ✓ **Vantaggi del cross calling**
>
> L'uso del cross calling dei costruttori:
>
> - riduce il **copia-incolla**;
> - rende il codice più **manutenibile**;
> - garantisce che tutti gli oggetti vengano inizializzati in modo coerente;
> - sfrutta pienamente il concetto di **overloading dei costruttori** visto nei paragrafi precedenti.

In questo modo, la classe offre più modalità di creazione degli oggetti, senza sacrificare chiarezza e qualità del codice.

I modificatori

Abbiamo visto che in Java esistono i **modificatori**, cioè parole chiave che descrivono le proprietà di un'entità (*classe, attributo o metodo*).
In particolare, i modificatori servono a definire tre aspetti fondamentali:

1. **la visibilità**
2. **la modificabilità**
3. **l'appartenenza alla classe o agli oggetti**

Modificatori di accesso (visibilità)

I **modificatori di accesso** regolano **chi può vedere e utilizzare una classe, un attributo o un metodo**.
Sono strettamente legati al concetto di **incapsulamento**.

I principali sono:

- `private`
  - **Utilizzato soprattutto per gli attributi**

    - **Un membro `private` è accessibile solo all'interno della classe in cui è dichiarato**
    - **È il modificatore più usato nella programmazione orientata agli oggetti perché favorisce l'incapsulamento**
- `public`
    - **Può essere applicato a classi, metodi e attributi**
    - **Un membro `public` è accessibile da qualsiasi altra classe**
    - In genere:
        - **le classi principali sono `public`**
        - **i metodi che rappresentano il comportamento "esterno" di un oggetto sono `public`**
- `protected`
  Rende visibile un membro:
    - **alle classi dello stesso package**
    - **alle sottoclassi, anche se si trovano in package diversi**

  È meno usato nelle applicazioni comuni ed è più frequente nello sviluppo di **framework o librerie**, dove l'ereditarietà gioca un ruolo centrale.
- **Visibilità di default (package-private)**
    - **Se non viene specificato alcun modificatore, il membro è visibile solo all'interno dello stesso package**
    - Anche questa modalità è poco usata nelle applicazioni commerciali, ma può risultare utile nella progettazione di librerie.

> ⓘ **In pratica, nello sviluppo di applicazioni "classiche", i modificatori più usati sono `private` e `public`, mentre `protected` e il default hanno un ruolo più di nicchia.**

Modificatore `final` (modificabilità)

Oltre alla visibilità, Java permette di controllare **se un'entità può essere modificata nel tempo**.

- `final` specifica che:
    - **un attributo non può cambiare valore dopo l'inizializzazione;**
    - **un metodo non può essere ridefinito (`overriding`) nelle sottoclassi;**
    - **una classe non può essere estesa.**

Nel caso degli attributi, `final` indica che:

- **lo stato dell'oggetto non evolve per quella proprietà.**
  Questo rafforza la **sicurezza e la coerenza del modello**.

Modificatore `static` (appartenenza alla classe)

Il modificatore `static` riguarda invece l'**appartenenza**:

- `static` specifica che un attributo o un metodo:
  - **appartiene alla classe**
  - **e non alle singole istanze (oggetti)**

Questo significa che:

- **esiste una sola copia del membro `static`;**
- **tale copia è condivisa da tutte le istanze della classe;**
- **può essere utilizzata senza creare un oggetto, accedendo direttamente alla classe.**

Uso combinato dei modificatori

I modificatori `static` e `final`:

- **possono essere usati insieme**
- **possono essere usati separatamente**
- **oppure non essere usati affatto**

Possono essere applicati a:

- **attributi**
- **metodi**
- **e, in alcuni casi, alle classi**

Questa combinazione consente di esprimere con precisione:

- **chi può accedere** a un membro,
- **se può cambiare nel tempo**,
- **a chi appartiene** (classe o oggetto).

Uso di `final`

Il modificatore `final` in Java può essere applicato a:

- **attributi,**
- **metodi**
- **classi,**
  ma il suo effetto varia a seconda del contesto.

1. Attributi `final`

- Se un attributo **primitivo** è `final`:
  - **il suo valore diventa costante e non può più essere modificato dopo l'inizializzazione**
    Esempio:

Java

```java
final int giorniSettimana = 7;
```

Se un attributo è un riferimento ad un oggetto e dichiarato `final`, il riferimento non può cambiare, cioè non può puntare a un altro oggetto.

⚠ **Tuttavia, lo stato interno dell'oggetto può comunque essere modificato:**

Java

```java
final List<String> nomi = new ArrayList<>();
nomi.add("Marco"); // OK, modifico l'oggetto
// nomi = new ArrayList<>(); // ERRORE, non posso cambiare il riferimento

```

2. Metodi `final`

Un **metodo** `final`:

- **non può essere ridefinito nelle sottoclassi.**
  Questo è utile per **bloccare comportamenti chiave** di una classe che non devono essere modificati da chi estende la classe:

Java

```java
public final void stampaSalario() {
    System.out.println(salario);
}
```

3. Classi `final`

Una classe `final` non può essere estesa.
Questo può servire per:

- motivi di **progetto** → **per evitare che qualcuno crei sottoclassi non previste,**
- o per motivi di **performance**→ p**erché il compilatore può ottimizzare alcune chiamate a metodi `final`**:

Java

```java
public final class Utility {
    public static double piGreco = 3.14159;
}
```

In sintesi:

- `final` sugli **attributi** → il valore o il riferimento non cambia
- `final` sui **metodi** → il comportamento è bloccato
- `final` sulle **classi** → la classe non può essere estesa

Questa distinzione è importante perché `final` **non significa sempre "costante" nel senso assoluto**, ma **"immutabile" rispetto alla struttura a cui viene applicato**.

Uso di `static`

Il modificatore `static` serve a indicare che:

- **un membro appartiene alla classe e non a una specifica istanza dell'oggetto.**
  Questo significa che **c'è una sola copia** dell'attributo o del metodo, condivisa tra tutte le istanze della classe.

1. Attributi `static`

- Un attributo `static`:
  - **è condiviso da tutti gli oggetti della classe.**
- **Non è necessario creare un oggetto per accedervi: si utilizza direttamente la classe.**
- Esempio:

Java

```java
public class Contatore {
    public static int totale = 0;

    public Contatore() {
        totale++; // Aggiorna il contatore globale
    }
}

Contatore c1 = new Contatore();
Contatore c2 = new Contatore();

System.out.println(Contatore.totale); // Output: 2
```

In questo esempio, `totale` è condiviso da tutti gli oggetti `Contatore`.
Non c'è una copia separata per `c1` o `c2`.

2. Metodi `static`

- Un metodo `static`:
  - **non richiede un oggetto per essere invocato.**
- All'interno di un metodo `static`:
  - **Non si può usare `this` (perché non c'è un oggetto associato)**
  - **Non si possono richiamare direttamente membri non statici**

Java

```java
public class MathUtil {
    public static int quadrato(int n) {
        return n * n;
    }
}

int risultato = MathUtil.quadrato(5); // Output: 25
```

3. Classi `static`

- In Java **solo le inner class possono essere dichiarate** `static`.
- Una inner class statica non è legata a un'istanza della classe esterna, quindi può essere istanziata senza creare un oggetto esterno.

In sintesi:

- `static` → appartiene alla classe, non agli oggetti
- Attributi statici → **una sola copia condivisa**
- Metodi statici → possono essere invocati **senza oggetto**
- Le inner class possono essere statiche → indipendenti dall'oggetto esterno

Costanti di classe (`static final`)

In Java, è possibile creare delle **costanti di classe**, cioè variabili **condivise da tutte le istanze** della classe e **immutabili**.
Per farlo, si combinano i modificatori `static` e `final`:

Java

```java
public class CostantiMatematiche {
    public static final double PI_GRECO = 3.14159;
    public static final int MAX_STUDENTI = 30;
}
```

Caratteristiche principali:

1. `static` → **appartiene alla classe e non agli oggetti, quindi tutte le istanze condividono lo stesso valore**.
2. `final` → **il valore non può essere modificato dopo l'inizializzazione.**
3. **Inizializzazione obbligatoria** → **la costante deve essere assegnata in fase di dichiarazione (non può essere lasciata vuota)**.
4. **Accesso tramite la classe** → non serve un oggetto per utilizzarla:

Java

```java
System.out.println(CostantiMatematiche.PI_GRECO); // Output: 3.14159
```

```
2  System.out.println(CostantiMatematiche.MAX_STUDENTI); // Output:
   30
```

Le costanti di classe sono molto utili per definire valori **universali e immutabili** all’interno di un programma, come limiti, parametri fissi o valori matematici.